Database Report on the Dollar Flu outbreak in Manhattan.

3428295

***Abstract*—This report will include information on the scenario, the process of creating the tables, importing the data, queries to find insights on the data and complexity.**

I. Introduction, Scenario and Data Collection

In 2001, a high-level bio-terror attack simulation named operation dark winter was conducted in Maryland, United States to model a fictional scenario detailing a smallpox attack on the country starting from Oklahoma City [1]. It aimed to raise awareness of the possibility of a biological attack, understand how overwhelmed emergency response would be and expose flaws in co-ordination among government officials.

The scenario is heavily inspired by the operation and will convey a snapshot covering a month after an outbreak of a genetically-engineered supervirus called The Dollar Flu in Manhattan, New York without quarantines or medical efforts. The virus is from the fictional work The Division by Tom Clancy which contains traits from several other viruses such as Ebola and Hantavirus. It has an infection rate of 95 percent and a recovery of 5 percent which will heavily influence the data for the snapshot[2]. The blocks of each neighborhood will be approximated.

To simulate the snapshot, data will be collected from open data sources detailing statistics on the populations of its neighborhoods. Sources will include data from the official government website [3] and neighborhood statistics from WorldAtlas [4]. Data for the number of infected and recovered cases among the population of each block will be generated based on the infection and recovery rate of the Dollar Flu (95 percent become infected, the rest recover).

II. Database Design and Decisions

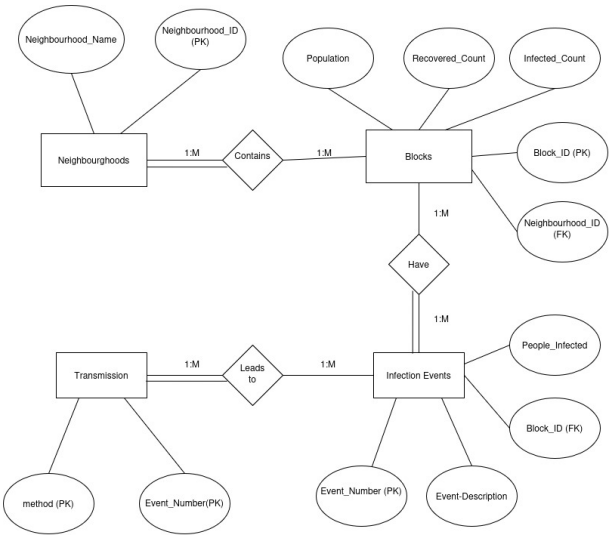


fig. 1. AN ER diagram of the database

The ER diagram visualizes our database and shows the decisions made. There will be 4 entities each represented by a single table. Each entity has its own primary key to make sure no duplicates exist and each entity (except transmission) will be in third normal form so every non key attribute is tied to the primary key. Neighborhood will have its own table rather than being part of the blocks table as this will remove the transitive dependency chain of block ID determining neighborhood ID determining neighborhood name. Transmission is its own table and works as a compound key as one event can have several different methods with one method per row leading to different combinations of event and transmission without repetition of one. Storing the transmission methods as a list for each event would break 1NF. The tables for cityblocks and infection events are all 3NF to make it obvious one table describes the blocks of the city and one describes the outbreaks, so that no mixed or unrelated attributes are included. All attributes for each table will include the NOT NULL option in sql so no record is incomplete.

III. Table Creation

We will first create the neighbourhood table. Neighbourhood ID will be used as the primary key to make sure each neighbourhood. The name of the neighbourhood will also be included:

```
    CREATE TABLE neighbourhood (
Neighbourhood_ID  INT  NOT NULL,
Neighbourhood_Name  varchar(50)  NOT NULL,
```

```
PRIMARY KEY(Neighbourhood_Id));
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	Neighbourhood_ID	int			No	None			Change Drop More
2	Neighbourhood_Name	varchar(50)	utf8mb4_0900_ai_ci		No	None			Change Drop More

fig. 2. The neighbourhood table with its primary key

To import data we will use the following code:

```
    -- data can only be loaded from
    wamp folder
LOAD DATA INFILE 'C:/wamp64/tmp/
Neighbourhoods.csv'
INTO TABLE neighbourhood
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;
```

This code will be used to import every single CSV onto the tables. It will look identical with the only difference being the name of the CSV. The names of the CSVs will be from the ER diagrams for each respective entity.

←T→	Neighbourhood_ID	Neighbourhood_Name
Edit Copy Delete	1	Harlem
Edit Copy Delete	2	Upper West Side
Edit Copy Delete	3	Upper East Side
Edit Copy Delete	4	Hell's Kitchen
Edit Copy Delete	5	Theater District
Edit Copy Delete	6	Midtown
Edit Copy Delete	7	Hudson Yards
Edit Copy Delete	8	Koreatown
Edit Copy Delete	9	NoMad
Edit Copy Delete	10	Chelsea
Edit Copy Delete	11	Flatiron district
Edit Copy Delete	12	Gramercy
Edit Copy Delete	13	Greenwich Village/West Village
Edit Copy Delete	14	East Village
Edit Copy Delete	15	SoHo
Edit Copy Delete	16	Lower east Side
Edit Copy Delete	17	Little Italy
Edit Copy Delete	18	Tribeca
Edit Copy Delete	19	Chinatown
Edit Copy Delete	20	Financial

fig. 3. neighbourhood data imported

The second table we will make is the city block table. NeighbourhoodID will be its foreign key and we will have blockID as its primary key and will also contain numbers on population as well as the number of people currently infected or recovered after 1 month.

```
    CREATE TABLE CityBlock (
Block_ID  INT  NOT NULL,
 Neighbourhood_ID  INT  NOT NULL,
Population  INT  NOT NULL,
 Infected_Count  INT  NOT NULL,
Recovered_Count  INT  NOT NULL,
 PRIMARY KEY (Block_ID),
 FOREIGN KEY(Neighbourhood_ID)
 REFERENCES neighbourhood
 (Neighbourhood_ID) );
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	Block_ID	int			No	None			Change Drop More
2	Neighbourhood_ID	int			No	None			Change Drop More
3	Population	int			No	None			Change Drop More
4	Infected_Count	int			No	None			Change Drop More
5	Recovered_Count	int			No	None			Change Drop More

fig. 4. cityblock table structure

We will create the infection event table. BlockID will act as the foreign key and each event will have its own number as its code along with an attribute describing the event and another for the number of people who were initially infected during after the event. We will do this with the following code:

```
CREATE TABLE InfectionEvent (
Event_Number  INT  NOT NULL,
 Block_ID  INT  NOT NULL,
People_Infected  INT  NOT NULL,
Event_Description  VARCHAR(255)  NOT NULL,
PRIMARY KEY (Event_Number),
FOREIGN KEY(Block_ID) REFERENCES
CityBlock (Block_ID) );
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	Event_Number	int			No	None			Change Drop More
2	Block_ID	int			No	None			Change Drop More
3	People_Infected	int			No	None			Change Drop More
4	Event_Description	varchar(255)	utf8mb4_0900_ai_ci		No	None			Change Drop More

fig. 5. result of creating infection event code

The following code will be used to create the transmission table:

```
CREATE TABLE transmission (
    Event_Number    INT  NOT NULL,
    Method VARCHAR(30)   NOT NULL,
    -- following will act as our compound key
    PRIMARY KEY (Event_Number, Method),
    FOREIGN KEY (Event_Number)
    REFERENCES InfectionEvent (Event_Number));
```

#	Name	Type	Collation	Attributes	Null	Default
1	Event_Number	int			No	None
2	Method	varchar(30)	utf8mb4_0900_ai_ci		No	None

fig. 6. above shows the result

IV. Insert Statements

This section will include examples of how we might want to insert new data into each of our tables. The statements are only there as examples and will insert instances only to show that the statements work. the data will be removed after.

We will use the following code to add a new neighbourhood:

```
   INSERT INTO neighbourhood(Neighbourhood_ID,
   Neighbourhood_Name)
VALUES (21, "random_neighbourhood");
-- The name is just an example since every
neighbourhood from manhattan is written down.
```

	Edit	Copy	Delete	19 Chinatown
	Edit	Copy	Delete	20 Financial
	Edit	Copy	Delete	21 random_neighbourhood

fig. 7. shows the new neighbourhood as an instance

For cityblock, we will insert a new block into the financial neighbourhood.

```
INSERT INTO cityblock(Block_ID,
Neighbourhood_ID,
Population, Infected_Count,
Recovered_Count)
VALUES (55, 20, 20000, 19000, 1000);
```

Edit Copy Delete	52	19	45220	42959	2261
Edit Copy Delete	53	20	16345	15528	817
Edit Copy Delete	54	20	19654	18671	983
Edit Copy Delete	55	20	20000	19000	1000

fig. 8. shows the new block inserted in the table

We will add a new infection outbreak for the event table with a description which is a realistic possibility in the setting:

```
INSERT INTO infectionevent (
Event_Number,
Block_ID, People_Infected, Event_Description)
VALUES (11, 19, 7, "Airborne particles
spread at a major museum exhibit");
```

Edit Copy Delete	9	41	7	Both infected and uninfected individuals are group...
Edit Copy Delete	10	16	5	Infected staff carry the virus into a retirement h...
Edit Copy Delete	11	19	7	Airborne particles spread at a major museum exhibi...

fig.9. shows the new event instance

If we wanted to add a new transmission method for an existing outbreak event we would use the following example:

```
INSERT INTO transmission(Event_Number, Method)
VALUES(2, "Shared Surfaces");
```

Edit Copy Delete	1	Contaminated Currency
Edit Copy Delete	1	Direct Contact
Edit Copy Delete	2	Airborne Particles
Edit Copy Delete	2	Shared Surfaces

fig. 10. An image showing the new instance of transmission method in the table

V. SQL Queries

A. Query 1

B. Query 2

C. Query 3

D. Query 4

E. Query 5

F. Query 6

VI. Conclusion

The conclusion goes here.

Appendix A
Proof of the First Zonklar Equation

Appendix one text goes here.

Appendix B

Appendix two text goes here.

Acknowledgment

The authors would like to thank...

References

[1] "Dark Winter Johns Hopkins Center for Health Security," *centerforhealthsecurity.org*. https://centerforhealthsecurity.org/our-work/tabletop-exercises/dark-winter-a-training-tabletop-exercise
[2] Green poison — the division wiki — fandom, https://thedivision.fandom.com/wiki/Green_Poison
[3] "NYC Population by Neighborhood — NYC Open Data," *Cityofnewyork.us*, 2018. https://data.cityofnewyork.us/City-Government/NYC-Population-by-Neighborhood/t79z-mgxr
[4] "Manhattan Neighborhoods by Population," *WorldAtlas*, Feb. 20, 2019. https://www.worldatlas.com/articles/manhattan-neighborhoods-by-population.html (accessed Oct. 25, 2026).

John Doe Biography text here.

Jane Doe Biography text here.